

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.		
Student Name:	PRITHIYANKA SHREE M	Reg. No.:	192521144
Section / Batch:	2025	Date:	29-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Q1 Set Partition Problem (NP-Complete / Backtracking Problem)

100 marks

Given a set of integers, divide it into two subsets with equal sum. Clearly define constraints related to subset formation and sum equality. Analyze how constraints impact the search space. Develop a backtracking solution to generate valid partitions. Apply pruning techniques to eliminate invalid subsets early. Justify how your solution ensures completeness and discuss computational complexity.

Code of Conduct

I PRITHIYANKA SHREE M(192521144) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: PRITHIYANKA SHREE M

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1 Set Partition Problem (NP-Complete / Backtracking Problem)

100 marks

Given a set of integers, divide it into two subsets with equal sum. Clearly define constraints related to subset formation and sum equality. Analyze how constraints impact the search space. Develop a backtracking solution to generate valid partitions. Apply pruning techniques to eliminate invalid subsets early. Justify how your solution ensures completeness and discuss computational complexity.

Your Code

```
1 n = int(input())
2 a = list(map(int, input().split()))
3
4 total = 0
5
6 for i in range(n):
7     total = total + a[i]
8
9 if total % 2 != 0:
10     print("No equal partition possible")
11 else:
12     target = total // 2
13     subset = []
14
15     def partition(index, current):
16         if current == target:
17             print("Subset 1:", end=" ")
18             for i in range(len(subset)):
19                 print(subset[i], end=" ")
20
21             print()
22             print("Subset 2:", end=" ")
23
24             for i in range(n):
25                 found = False
26
27                 for j in range(len(subset)):
28                     if a[i] == subset[j]:
29                         found = True
30                         break
31
32                 if found == False:
33                     print(a[i], end=" ")
34
35             print()
36             return True
37
38         if current > target:
39             return False
40
41         if index == n:
42             return False
43
44         # Include current element
45         subset.append(a[index])
46
47         if partition(index + 1, current + a[index]):
48             return True
49
50         subset.pop()
51
```

```
51
52     # Exclude current element
53     if partition(index + 1, current):
54         return True
55
56     return False
57
58 if partition(0, 0) == False:
59     print("No equal partition possible")
```

INPUT AND OUTPUT:

Input

```
4
1 5 11 5
```

Output

```
Subset 1: 1 5 5
Subset 2: 11
```
